COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	07/23/2026	**DATE SUBMITTED:**	07/28/2026
NAME:	Pangilinan Jr., Bienvenido B.	**INSTRUCTOR:**	Engr. Jimlord Quehado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 3-1. Output of Initial/Simple Implementation

Screenshot	
Discussion	Based on my observation the program demonstrates the linked list at first it initialize the 6 **pointers** to **null** then next, it allocates memory for each node on the heap using new. Finally, it assigns character values to the nodes and links them together using next pointers, ending the list with **nullptr**

Table 3-2. Code for the List Operations

Operation	Screenshot

Traversal	`// printing out every list`
	`std::cout << "Linked list traversal: ";`
	`listTraversal(head);`

Insertion at head	`//inserting new head`
	`std::cout << "Newly inserted head of Linked List: ";`
	`sllinsterHead( newData: 0, &head);`
	`listTraversal(head);`

Insertion at any part of the list	`//inserting new head`
	`std::cout << "Newly inserted head of Linked List: ";`
	`sllinsterHead( newData: 0, &head);`
	`listTraversal(head);`

Insertion at the end	`//inserting at the end`
	`std::cout << "Newly Inserted at the end: \n";`
	`sllinsertEnd( newData: 10,&head);`
	`listTraversal(head);`

Deletion of a node	`//deleting a node by finding the data`
	`std::cout << "Deleted a node: \n";`
	`sllDelete( findData: 10, &head);`
	`listTraversal(head);`

Table 3-3. Code and Analysis for Singly Linked Lists

a.	**Source Code**	`// printing out every list`
		`std::cout << "Linked list traversal: ";`
		`listTraversal(head);`

	Console	`Linked list traversal: 1->2->4`
		
		`Process finished with exit code 0`

b.	**Source Code**	`//inserting new head`
		`std::cout << "Newly inserted head of Linked List: ";`
		`sllinsterHead( newData: 0, &head);`
		`listTraversal(head);`

	Console	`Newly inserted head of Linked List: 0->1->2->4`
		
		`Process finished with exit code 0`

c.	**Source Code**	`std::cout << "Newly inserted after a node of Linked List: \n";`
		`//inserted after the  VVVV`
		`sllGeneralinsertion( newData: 3,head->next);`
		`listTraversal(head);`

	Console	`Newly inserted after a node of Linked List:`
		`0->1->3->2->4`
		
		`Process finished with exit code 0`

d.	**Source Code**	`//inserting at the end`
		`std::cout << "Newly Inserted at the end: \n";`
		`sllinsertEnd( newData: 10,&head);`
		`listTraversal(head);`

	Console	`Newly Inserted at the end:`
		`0->1->3->4->2->4->10`
		
		`Process finished with exit code 0`

e.	**Source Code**	`//deleting a node by finding the data`
		`std::cout << "Deleted a node: \n";`
		`sllDelete( findData: 10, &head);`
		`listTraversal(head);`

	Console	`Deleted a node:`
		`0->1->3->4->2->4`
		
		`Process finished with exit code 0`

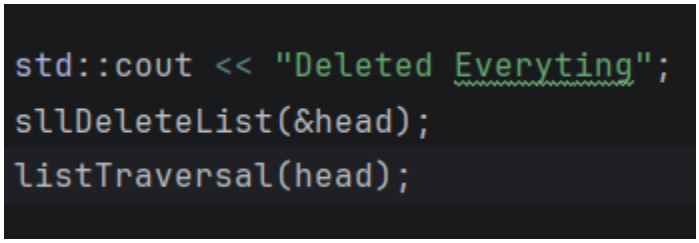
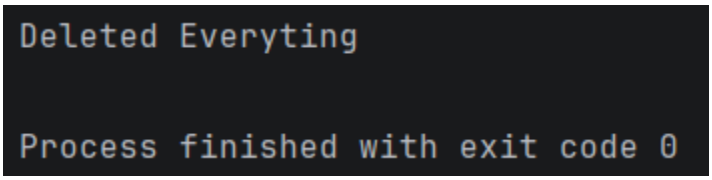
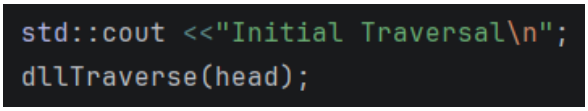
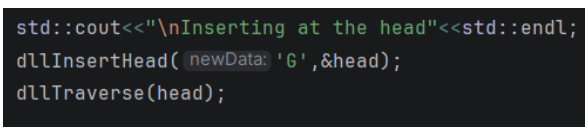
f.	**Source Code**	
	Console	

Table 3-4. Modified Operations for Doubly Linked Lists

Screenshots(s)	Analysis
	When creating three nodes containing 'C', 'P', and 'E' and connecting them using **next** and **prev** pointers when **dllTraverse()** runs, it prints the nodes moving forward using **next**. Once it hits the end of the list, it uses the last node to walk backward using **prev** in which results in printing forward as C P E and then reversing to E P C.
	Calling **dllInsertHead('G', &head)** creates a new node **G** and puts it at the very front as the new head. Its next pointer connects forward to the old head **C**, and the **C** prev pointer connects backward to **G** The head pointer then updates to point to **G**, which results to printing forward as G C P E and then reversing to E P C G.
	The function added new data which is **1** as it is new tail in the end which the **1** loops through the list from head until it reaches the last node E. It connects the next pointer forward to the new node 1 and is which, which results in printing forward as G C P E 1 and then reversing to 1 E P C G.
	This function creates another **G** as its new head node and place it from the front linking it to the old **G** node and the output is, GGCPE1 and reverse it to 1EPCGG

`std::cout<<"\nInserting at the any node"<<std::endl;`
`//insert 'X' after 'P'`
`DoubleList<char>* current = head;`
`while(current != nullptr && current->data != 'P'){`
`  current = current->next;`
`}`
`if(current != nullptr){`
`  dllInsertAny( newData: 'X',current);`
`}`
`dllTraverse(head);`
	The **while** loop searches through the list finding the node **P,** once that node is found, the **X** node squeezes through and inserted right after the **P,** This adds **X** between **P** and **E**, which results to printing forward as G G C P X E 1 and then reversing to 1 E X P C G G.

`std::cout << "\nDeleting entire list:\n";`
`dllDeleteList(&head);`
`dllTraverse(head);`
	When the function **dllDeleteList,** and goes every node clearing them one by one then frees up the memory and it set to the **nullptr**. To prevent memory leaks, so when you run it again both forward and reverse is empty since the all node is cleared out

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.
Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing
Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.
These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.
We'll take the use case of a music player. It should have following functions supported:
Create a playlist using multiple songs.
24
Add songs to the playlist.
Remove a song from the playlist.
Play songs in a loop (for this activity, we will print all the songs once).
Here are the steps to solve the problem:
Design the basic structure that supports circular data representation.
After that, implement the insert and delete functions in the structure.
Implement a function for traversing the playlist.

ILO B: Solve given problems utilizing linked lists in C++

Source Code

```cpp
#include <iostream>
#include <string>
#include "double_ll_h_2.h"

int main() {
    DoubleList<std::string> *playlist = nullptr;

    // 1. Create a playlist using multiple songs
    circularInsertTail( newData: std::string( s: "Leonara"), &playlist);
    circularInsertTail( newData: std::string( s: "About You"), &playlist);
    circularInsertTail( newData: std::string( s: "KLWKN"), &playlist);
    std::cout << "My Playlist:\n";
    printPlaylist(playlist);
    std::cout << "\n";

    // 2. Insert at the end
    std::cout << "Inserting at the end:\n";
    circularInsertTail( newData: std::string( s: "Heaven Knows"), &playlist);
    printPlaylist(playlist);
    std::cout << "\n";

    // 3. Delete a node
    std::cout << "Deleting a node:\n";
    circularDeleteSong( targetData: std::string( s: "KLWKN"), &playlist);
    printPlaylist(playlist);
    std::cout << "\n";

    // 4. Play Next / Previous Song
    DoubleList<std::string> *current = playlist;

    std::cout << "Play Next Song:\n";
    playNextSong(&current);
    std::cout << "\n";

    std::cout << "Play Previous Song:\n";
    playPreviousSong(&current);

    return 0;
}
```

H File

```cpp
#ifndef HOA3_1_DOUBLE_LL_H_2_H
#define HOA3_1_DOUBLE_LL_H_2_H

#include ...

template<typename T>
class DoubleList {
public:
    T data;
    DoubleList<T>* next = nullptr;
    DoubleList<T>* prev = nullptr;
};

template<typename T>
void circularInsertTail(T newData, DoubleList<T>** head) {
    DoubleList<T>* newNode = new DoubleList<T>;
    newNode->data = newData;

    if (*head == nullptr) {
        *head = newNode;
        newNode->next = newNode;
        newNode->prev = newNode;
        return;
    }

    DoubleList<T>* tail = (*head)->prev;

    tail->next = newNode;
    newNode->prev = tail;
    newNode->next = *head;
    (*head)->prev = newNode;
}

template<typename T>
void printPlaylist(DoubleList<T>* start) {
    if (start == nullptr) {
        std::cout << "Playlist is empty!\n";
        return;
    }

    DoubleList<T>* current = start;
    do {
        std::cout << current->data << "->";
        current = current->next;
    } while (current != start);
```

```
    do {
        if (current->data == targetData) {
            toDelete = current;
            break;
        }
        current = current->next;
    } while (current != *head);

    if (toDelete == nullptr) {
        std::cout << "Song '" << targetData << "' not found in playlist.\n";
        return;
    }

    if (toDelete->next == toDelete) {
        delete toDelete;
        *head = nullptr;
        std::cout << "Removed '" << targetData << "'. Playlist is now empty.\n";
        return;
    }

    toDelete->prev->next = toDelete->next;
    toDelete->next->prev = toDelete->prev;

    if (toDelete == *head) {
        *head = toDelete->next;
    }

    delete toDelete;
    std::cout << "Successfully removed song: " << targetData << "\n";
}

#endif //HOA3_1_DOUBLE_LL_H_2_H
```

Output

```
My Playlist:
Leonara->About You->KLWKN->END

Inserting at the end:
Leonara->About You->KLWKN->Heaven Knows->END

Deleting a node:
Successfully removed song: KLWKN
Leonara->About You->Heaven Knows->END

Play Next Song:
Now playing from the Playlist: About You
About You->Heaven Knows->Leonara->END

Play Previous Song:
Now playing from the Playlist: Leonara
Leonara->About You->Heaven Knows->END

Process finished with exit code 0
```

Analysis

As we created the file, we created a playlist with songs like Leonara, About You, and KLWKN. Instead of stopping at a **null pointer** like a standard linked list, this list forms a continuous **loop** in memory. The last song points its next **pointer** back to the first song, and the first song points its **prev** pointer backward to the last song. This allows the playlist to cycle continuously and when we print the playlist using the **printPlaylist** the output displays and END in it (the end function is not in the node or memory but sort of marker because the list is in circular loop that stops once it completes one full circle back to the starting song, printing "END" just to show that one full loop was completed on screen. A new song like Heaven Knows using **circularInsertTail**, the code finds the current end of the list using **playlist->prev**. It places Heaven Knows right between the last song and the first song, updating all of them with the heaven knows song. When removing a specific song like KLWKN using **circularDeleteSong**, the code searches through the list to find the target node and removes it. Once it got removed, the node freed up computer memory, preventing memory leaks while keeping the rest of the playlist linked and functional.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.